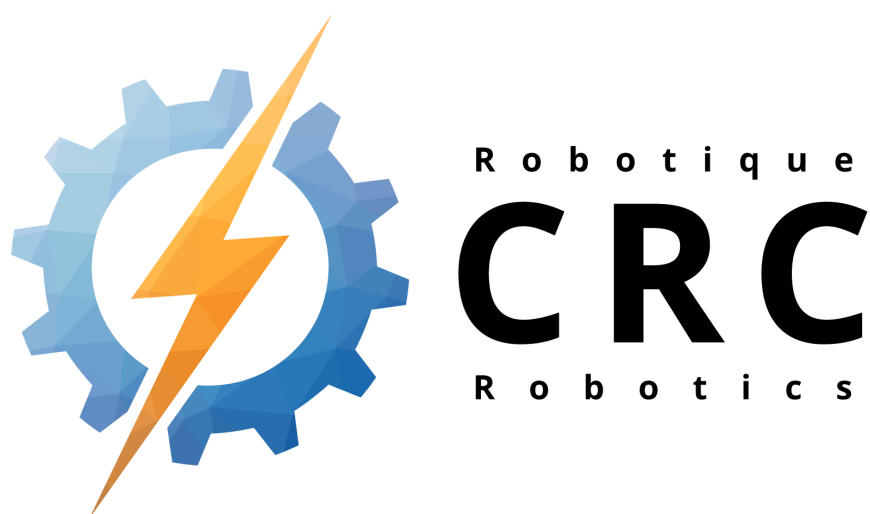




CRC
**COMPÉTITION DE
PROGRAMMATION**



**LIVRET DE PROBLÈMES
BLOC A**

QUELQUES NOTES

- Les règles complètes sont dans la section 4 du livret des règlements. **N'oubliez pas les multiplicateurs!**
- Vous avez jusqu'à **20h29** pour remettre vos solutions. Remettez la structure de fichier comme vous l'avez reçu avec vos solutions dans les fichiers .py associés.
- N'hésitez pas à nous poser des questions et à communiquer avec les membres de votre équipe de programmation. Ce n'est pas un examen!
- **On vous donne des templates faciles à utiliser pour votre code. Vous devez les utiliser comme des templates similaires sont utilisés pour la correction!**

UTILISATION DU FICHIER MODÈLE

- Tous les fichiers possèdent une fonction solve() dans lequel vous devez résoudre le problème. **NE CHANGEZ PAS LE NOM DE CETTE FONCTION!**
- Pour rouler vos tests, dans un terminal (nous vous conseillons l'utilisation de vscode) écrivez la ligne de commande: `pytest` pour rouler les tests. Si vous faites la commande `pytest` dans le répertoire contenant tous les fichiers, tous les tests seront exécutés.
- Pour rouler les tests d'un problème spécifique, naviguez dans le répertoire de la question et exécutez la commande `pytest` pour rouler les tests du problème. Vous pouvez faire la même chose pour une section entière!

STRUCTURE

Chaque problème contient une petite mise en situation comme celle-ci expliquant les fondements du problème et donnant les bases nécessaires pour résoudre celui-ci. Chaque problème contient aussi la répartition des points pour le problème.

Spécifications d'entrée et de sortie:

Dans cette section, on retrouve les caractéristiques des entrées qui peuvent être fournies au code en question ainsi que les critères attendus pour les sorties du programme.

Exemple d'entrée et de sortie:

Dans cette section se trouve un exemple d'entrée (parfois constitué lui-même de plusieurs sous-exemples) pour que vous puissiez tester votre code. L'exemple de sortie donne donc la réponse attendue pour cette entrée.

Explication de la première sortie:

Si le problème n'est toujours pas clair après la mise en situation, l'explication de la première sortie sert parfois à démêler le tout en expliquant comment la première entrée est traitée et en montrant le chemin menant à cette réponse.

Table des matières

PROBLÈMES 2D (2D)	4
2D1: Carrés imbriqués (15 points)	4
2D2: Bibliothèque (35 points)	6
2D3: Gros bruit (75 points)	9
ORGANISATION DE DONNÉES (DS)	12
DS1: Le cadeau parfait! (30 points)	12
DS2: Relais de paquets de données (35 points)	14
DS3: Liste chaînée simple (15 points)	17
JEUX VIDÉOS (VG)	19
VG1: Overcooked! (30 points)	19
VG2: Collisions (35 points)	22
VG3: Pac-Man (40 points)	24
VG4: Tetris (45 points)	27
TRAITEMENT DE TEXTE (TP)	30
TP1: Rangées et colonnes (40 points)	30
TP2: Haïku (20 points)	32
TP3: Palindroming (55 points)	34
TP4: Pangramme (35 points)	36

A diagram showing a large rectangle with a smaller rectangle inside it. The large rectangle is labeled with '1' on the left and '2' on the right. The smaller rectangle is labeled with '3' on the left and '4' on the right. The smaller rectangle is divided into four quadrants by a horizontal and a vertical line.

Exemple de sortie:

254

152

166

Explication de la première sortie:

Dans le premier exemple, nous avons un mur d'une largeur de 22 caractères, ce qui nous donne une largeur de 22 pieds. Pour la hauteur, nous avons 7 caractères, ce qui correspond à 14 pieds. La surface totale du mur en incluant la fenêtre est de 308 pieds carrés.

La fenêtre est d'une largeur de 9 caractères, donc 9 pieds et d'une hauteur de 3 caractères donc 6 pieds. Ceci nous donne une surface de 54 pieds carrés. La différence des deux surface nous donne une surface finale de $308 - 54 = 254$ pieds carrés.

2D2: Bibliothèque (35 points)

Vous voulez commencer une bibliothèque virtuelle et souhaitez faire l’affichage des livres sur votre étagère. Pour rendre l’arrangement agréable, vous choisissez de mettre les livres du plus grand à gauche, au plus petit à droite.

La grandeur de vos livres affichés est déterminée par la place nécessaire pour afficher entièrement le livre sur la reliure. Par contre, pour les livres d’une même grandeur, vous devrez placer les livres en ordre alphabétique.

Pour faire votre couverture de livre, le dessus du livre est le suivant: " , le bas du livre, " , et pour la section avec les lettres, " | X | ". Donc, on a le côté du livre, un espace, la lettre, un espace puis le côté du livre.

Spécifications d’entrée:

En entrée vous recevrez:

- **books:** un *array* de *string* qui sont les titres des livres sur l’étagère.

Spécifications de sortie:

En sortie vous devrez fournir:

- un *array* de *string* qui représente le visuel de l’étagère de livres.

Exemple d’entrée:

```
["Hobbit", "Disney", "Inferno", "Narnia", "Wicked", "Harry Potter"]
```

```
["The Hunger Games", "The Great Gatsby", "The book thief", "The Outsiders", "The little Prince", "Pride and Prejudice"]
```

```
["1984", "Twilight", "The Giver", "Divergent", "Holes", "The Host", "Uglies", "Eragon"]
```

Exemple de sortie:

H a r r y					
P o t t e r	I n f e r n o	D i s n e y	H o b b i t	N a r n i a	W i c k e d

P r i d e					
a n d	T h e	T h e	T h e	T h e	T h e
P r e j u d i c e	l i t t l e	G r e a t	H u n g e r	b o o k	O u t s i d e r s
	P r i n c e	G a t s b y	G a m e s	t h i e f	

D i v e r g e n t	T h e	T h e	T w i l i g h t				
	G i v e r	H o s t		E r a g o n	U g l i e s	H o l e s	1 9 8 4

Explication de la première sortie:

Dans le premier exemple, on commence par trouver les livres du plus long au plus court pour obtenir notre ordre de grandeur. Une fois que c'est fait, on s'assure que les titres de la même longueur sont classés par ordre alphabétique. Nous obtenons la liste suivante:

Harry Potter

Inferno

Disney

Hobbit

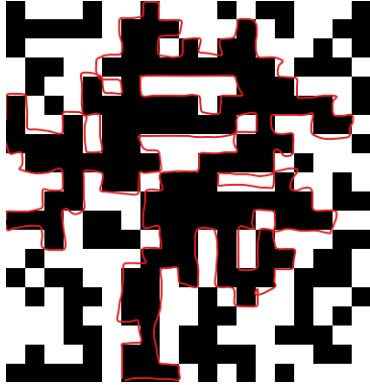
Narnia

Wicked

Il ne reste plus qu'à afficher la liste comme des reliures de livres sur une étagère!

2D3: Gros bruit (75 points)

Vous devrez calculer le périmètre du plus gros bloc de bruit – donc celui avec le plus grand périmètre – sur un plateau donné. Un bloc de bruit est un ensemble de pixels de bruit connectés orthogonalement (pas par diagonales, les pixels doivent partager un côté). Voici un exemple:



N'oubliez pas de prendre en compte le périmètre intérieur s'il y en a un.

Spécifications d'entrée:

En entrée vous recevrez:

- un *array* de *strings* avec des “.” pour représenter les cases vides et des “#” pour représenter les cases pleines

Spécifications de sortie:

Un *int* qui représente le périmètre total du plus gros bloc de bruit (extérieur ET intérieur)

Exemple d'entrée:

```
[ ".....##.....",
  "##.....#####",
  "###...###.#####",
  ".##...##.#.##...#",
  "#####.#...#.#...##",
  "#####.##.#...###",
  "...##.##.....##.",
  "...#.#####...#####",
  "##.#.#####.##.##.",
  ".#...##.##...#.#...#",
  "...##.###...####.##.",
  "...##.#####.....##.",
  ".#...##...##.....#",
  "...#.#.#.#.....",
  ".##...#.#.#.....##.",
  ".#...#.....#.#.##.",
  "#...##.....#####.#",
  "#####...###.#####",
  ".##...#.#.##...#",
  ".###.###.###....." ]
```

```
[ ".#.#.#.#.#.#.#",
  ".....###.#####",
  ".#.....###.###.",
  ".##.#.#.#.#.#.#",
  "####.###.....#####",
  "##.....#.#.#",
  "....#.#.#.#.#.#",
  "...###.#####.#.#",
  ".#.#####.##...",
  ".#..##.....#.#.",
  ".....#.#.###",
  "#.....#.#",
  ".....#.#.#.#.#",
  "...#####.##.#",
  "####.###.##.#...",
  ".#.....#.#.#...",
  ".#.....#####.##",
  "###.##.##.###.##",
  ".#.#.#.#.#.#.#",
  "...##.....###.##" ]
```

```
[ ".##.##.##.##.##.",
  "##.###.....#####.",
  "...##.#.#.#.#.#",
  "...##.#.#.#.#.#",
  "#...#..###.....#.#",
  ".....#.#.#.#.#",
  ".....#.#.#.#.#",
  "###.#####.#####.",
  "##.#####.###.###",
  "...##.##.#.#.#.",
  "...#.#.###.##.....",
  "...#.#.###.###.##.",
  "#####.###.###.#",
  ".###.....###.###.",
  ".##...#.#.##.##.#",
  "#...###.#.#.###.",
  ".##.###.##.##.##.",
  ".##.#.#.#.#.##.",
  "#####.###.###.#",
  ".###.....###.###.",
  ".##...#.#.##.##.#",
  "#...###.#.#.###.",
  ".##.###.##.##.##.",
  ".##.#.#.#.#.##.",
  ".....#.#.#####.#." ]
```

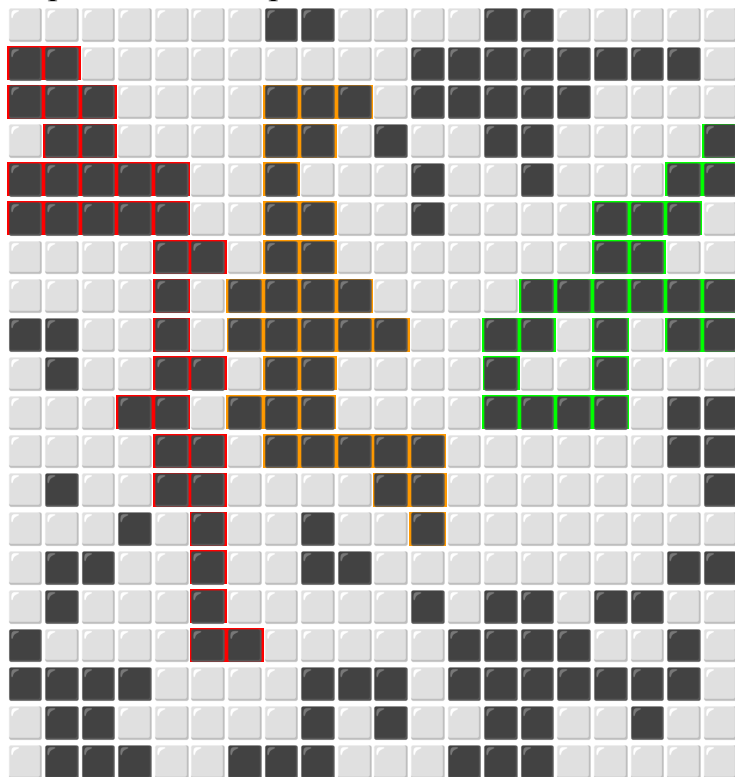
Exemple de sortie:

54

84

104

Explication de la première de sortie:



Périmètres respectifs des gros blocs de bruits dans le premier exemple:

Bloc rouge: 54, Bloc orange: 46, Bloc vert: 44

Donc, nous donnons le périmètre du bloc rouge.

ORGANISATION DE DONNÉES (DS)

DS1: Le cadeau parfait! (30 points)

Le Père Noël vous a demandé de l'aide pour organiser la distribution de cadeaux pour l'année prochaine. Pour chaque enfant, il veut choisir uniquement les jouets qui lui conviennent selon son âge et ses intérêts.

Chaque jouet est décrit par la structure suivante :

- **id** : identifiant du jouet (string)
- **minAge** : âge minimal recommandé (inclusivement, int)
- **maxAge** : âge maximal recommandé (inclusivement, int)
- **category** : catégorie du jouet (string)
- **price** : prix de fabrication du jouet (int)

Règles de sélection:

1. l'âge donné est compris entre **minAge** et **maxAge** inclusivement
2. la catégorie du jouet est celle voulue

Les jouets valides doivent être triés par ordre de prix croissant et la solution doit retourner un *array* contenant les ID des jouets.

Spécifications d'entrée:

En entrée vous recevrez:

- **toys**: un *array* de *ToyInfo* qui sont les jouets avec leur caractéristiques.
- **age**: un *int* de l'âge de l'enfant
- **category**: une *string* de la catégorie de jouet qui intéresse l'enfant

Spécifications de sortie:

En sortie vous devrez fournir:

- un *array* contenant en *string* les ID des jouets choisis en ordre de prix du moins cher au plus cher.

Exemple d'entrée:

```
toys = [  
    ToyInfo(3, 6, Category.PUZZLE, 15, 101, "Animal Puzzle"),  
    ToyInfo(5, 9, Category.FIGURINE, 20, 102, "Knight Figure"),  
    ToyInfo(6, 12, Category.PUZZLE, 10, 103, "World Map Puzzle"),  
    ToyInfo(4, 7, Category.CAR, 18, 104, "Race Car"),]  
age = 6  
category = Category.PUZZLE
```

```
toys = [  
    ToyInfo(7, 12, Category.BOARD_GAME, 35, 201, "Treasure Island"),  
    ToyInfo(6, 10, Category.BOARD_GAME, 25, 202, "Math Challenge"),  
    ToyInfo(8, 14, Category.COLLECTIBLE, 40, 203, "Rare Cards"),  
    ToyInfo(3, 7, Category.BOARD_GAME, 30, 204, "Family Fun"),]  
age = 8  
category = Category.BOARD_GAME
```

```
toys = [  
    ToyInfo(10, 16, Category.VIDEO_GAME, 60, 301, "Space  
Adventure"),  
    ToyInfo(8, 12, Category.VIDEO_GAME, 45, 302, "Puzzle Quest"),  
    ToyInfo(12, 18, Category.VIDEO_GAME, 70, 303, "Battle Arena"),  
    ToyInfo(9, 14, Category.CAR, 40, 304, "Remote Racer"),]  
age = 11  
category = Category.VIDEO_GAME
```

Exemple de sortie:

[103, 101]

[202, 201]

[302, 301]

Explication de la première sortie:

Dans le premier exemple, on conserve seulement Animal Puzzle et World Map puzzle comme ce sont les deux seuls *puzzles* (la catégorie demandée). On regarde par la suite les restrictions d'âge et on remarque que les deux sont valides pour un enfant de 6 ans.

Avec nos deux jeux valides, on peut maintenant les classer en ordre de prix. Ceci nous laisse avec l'ordre World Map Puzzle, qui est le moins cher (ID: 103), suivi de Animal Puzzle, qui est le plus cher des deux (ID: 101).

DS2: Relais de paquets de données (35 points)

Un réseau est constitué de N machines (numérotées de 1 à N), chacune exécutant son propre serveur local. Chaque machine a un seul paquet sortant en attente: la machine i a l'intention d'envoyer son paquet à la machine a_i , où $a_i \neq i$.

Un ordre de planification est défini par une permutation $(p_1, p_2, \dots, p_N)$ de $1 \dots N$. Les transmissions sont ensuite effectuées séquentiellement comme suit.

Pour chaque i de 1 à N :

- Si le destinataire prévu, la machine a_{p_i} , a déjà arrêté son serveur (parce qu'il a déjà envoyé son propre paquet), alors la transmission de la machine p_i est abandonnée et celle-ci reste inactive.
- Sinon, la machine p_i livre avec succès son paquet au serveur de la machine a_{p_i} , transférant v_{p_i} *bytes* de données utiles. Immédiatement après l'envoi, la machine p_i arrête son propre serveur.

Déterminez la quantité maximale de données utiles (en *bytes*) qui peut être livrée avec succès sur toutes les machines, en tenant compte de tous les ordres de planification possibles.

Indice:

Vous pouvez supposer qu'il y a au moins deux machines dans le réseau et qu'aucune machine n'essaie d'envoyer un paquet à elle-même (c'est-à-dire $a_i \neq i$ pour tous les i).

Une permutation de $1 \dots N$ signifie un réarrangement de l'ordre des nombres de 1 à N .

Par exemple, si $N = 3$, alors $(1, 2, 3)$, $(2, 3, 1)$ et $(3, 1, 2)$ sont des permutations valides, tandis que $(1, 2, 2)$ et $(4, 1, 3)$ ne le sont pas.

Spécifications d'entrée:

En entrée vous recevrez:

- N , un *entier*, qui désigne le nombre de machines dans le réseau.
- a , un *array* de N *entiers*, où a_i désigne la machine cible à laquelle le paquet de la machine i est destiné.
- v , un *array* de N *entiers*, où v_i désigne la taille de la charge utile en *bytes* que la machine i livrera après une transmission réussie.

Spécifications de sortie:

En sortie vous devrez fournir:

- un seul *entier*, qui désigne la quantité maximale de charge utile (en *bytes*) pouvant être transmise sur toutes les machines, pour tous les ordres de planification possibles p .

Exemple d'entrée:

$n = 4$

$a = [2, 4, 1, 3]$

$v = [20, 60, 40, 80]$

$n = 2$

$a = [2, 1]$

$v = [100, 50]$

$n = 5$

$a = [2, 1, 4, 5, 3]$

$v = [10, 30, 20, 40, 15]$

Exemple de sortie:

180

100

90

Explication de la première sortie:

Considérons l'ordre de planification $p = (3, 1, 2, 4)$:

- La machine 3 envoie son paquet au serveur de la machine 1, livrant 40 *bytes*.
- La machine 1 envoie son paquet au serveur de la machine 2, livrant 20 *bytes*.
- La machine 2 envoie son paquet au serveur de la machine 4, livrant 60 *bytes*.
- La machine 4 tente d'atteindre la machine 3, mais celle-ci s'est déjà arrêtée. La transmission est interrompue.

Le total ici est de $40 + 20 + 60 = 120$ *bytes*.

Considérons maintenant l'ordre de planification $p = (2, 4, 3, 1)$:

- La machine 2 envoie son paquet au serveur de la machine 4, livrant 60 *bytes*.

- La machine 4 envoie son paquet au serveur de la machine 3, livrant 80 *bytes*.
- La machine 3 envoie son paquet au serveur de la machine 1, livrant 40 *bytes*.
- La machine 1 tente d'atteindre la machine 2, mais celle-ci s'est déjà arrêtée. La transmission est interrompue.

Le total ici est de $60 + 80 + 40 = 180$ *bytes*. Il est possible de prouver qu'aucun ordre de planification n'atteint un total supérieur à celui-ci.

DS3: Liste chaînée simple (15 points)

Les listes chaînées sont extrêmement utiles en programmation mais elles sont très peu discutées. Les listes chaînées sont simples et ont un avantage par rapport aux *arrays* parce qu'elles n'ont pas de taille fixe et il est super facile d'ajouter un élément à n'importe quel endroit sans difficulté ou sans avoir à tout déplacer pour garder le même ordre.

Leur fonctionnement est relativement simple et chaque unité de la liste chaînée contient 2 éléments. Le premier est l'information à stocker et le deuxième est un pointeur à la position de l'élément suivant.

Les éléments de la liste chaînée peuvent donc être stockés n'importe où dans l'ordinateur. Il faut suivre la chaîne de pointeurs pour traverser les éléments. Nous allons vous donner une liste chaînée dont les éléments seront répartis un peu au hasard dans un *array*. Vous devrez nous fournir la séquence des éléments selon l'ordre de la liste chaînée. Le dernier élément de la liste chaînée va pointer vers la position impossible -1.

Spécifications d'entrée:

En entrée vous recevrez:

- **memory**: un *array* d'éléments de liste chaînée qui contient comme information des *int*
- **start**: la position du premier élément de la liste chaînée dans l'*array* d'entrée

Spécifications de sortie:

En sortie vous devrez fournir:

- un *array* du contenu des éléments (int) dans l'ordre de la liste chaînée

Exemple d'entrée:

```
[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]
start: 0
```

```
[(5, -1), (2, 2), (58, 0), (61, 1), (77, -1)]
start: 3
```

```
[(49, 2), (90, 0), (1, -1), (44, 6), (26, 4)]
start: 1
```

Exemple de sortie:

```
[17, 1, 14, 22]
```

```
[61, 2, 58, 5]
```

```
[90, 49, 1]
```

Explication de la première sortie:

Dans le premier exemple, on a les cases mémoires suivantes:

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Comme on commence à la position 0, on prend l'élément 17 pour ajouter à notre liste chaînée.

Le pointeur de la case 0 nous pointe vers la case 2.

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Nous avons ici la valeur 1 comme élément de liste et le pointeur vers la case 1.

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Nous avons la valeur 14 et le pointeur vers la case 3.

[(17, 2), (14, 3), (1, 1), (22, -1), (31, 5), (2, 4)]

Nous avons la valeur 22 et un pointeur vers -1 qui n'est pas une position valide, donc nous avons atteint le dernier élément de notre liste chaînée. Ceci nous donne le résultat suivant:

[17, 1, 14, 22]

JEUX VIDÉOS (VG)

VG1: Overcooked! (30 points)

Vous jouez à Overcooked avec votre meilleur(e) ami(e) en train d'en arracher sur un niveau particulièrement difficile, tellement que vous commencez à vous demander si le niveau est même théoriquement possible. Vous cherchez donc à calculer le profit maximum qu'il est possible d'atteindre dans ce niveau et, pour ce faire, vous décidez d'élaborer un programme qui calcule le tout.

Ceci étant dit, il vous faudra commencer relativement simple. C'est pourquoi nous ne considérerons pas les distances à parcourir entre les différentes tâches à effectuer dans la cuisine et le temps requis pour de tels déplacements (ou ceci deviendrait un problème d'optimisation plutôt complexe!). Une série d'ingrédients vous sera donnée en entrée correspondant à la recette à effectuer et vous devrez vendre le plus de recettes complétées possible dans le temps donné (**il ne faut pas oublier que vous disposez de deux joueurs pouvant effectuer des tâches en même temps!**) afin de maximiser vos profits. Chaque ingrédient dans la recette peut être précédé d'un adjectif (**et d'un seul!**) indiquant comment il doit être préparé. Les adjectifs possibles sont:

- **<none>** c'est-à-dire que l'ingrédient ne sera pas précédé d'un adjectif et ne consistera que d'un mot. Un **ingrédient non préparé** nécessite **1 seconde du temps d'un joueur** pour l'ajouter à la recette.
- **"cut"**: cet ingrédient doit être **coupé** sur une planche à découper. Couper un ingrédient et l'ajouter à la recette nécessitent **5 secondes du temps d'un joueur**. Il y a assez de planches à découper à disposition pour que les deux joueurs coupent en même temps.
- **"boiled"**: cet ingrédient doit être **bouilli** dans un chaudron. Un ingrédient prend **10 secondes à bouillir mais ne nécessite qu'1 seconde du temps d'un joueur** pour l'ajouter à la recette, c'est-à-dire qu'un chaudron est occupé pendant 10 secondes, mais ces 10 secondes ne comptent pas dans le temps des joueurs. Vous disposez de **4 chaudrons** pouvant bouillir indépendamment dans votre cuisine.
- **"cooked"**: cet ingrédient doit être **cuit** dans une poêle. Un ingrédient prend **7 secondes à cuire mais ne nécessite qu'1 seconde du temps d'un joueur** pour l'ajouter à la recette, c'est-à-dire qu'une poêle est occupée pendant 7 secondes, mais ces 7 secondes ne comptent pas dans le temps des joueurs. Vous disposez de **2 poêles** pouvant cuire indépendamment dans votre cuisine.

En plus de tous les ingrédients, il faudra **soustraire 1 seconde additionnelle du temps d'un des joueurs par recette** pour prendre l'assiette dans laquelle sera servie la recette. Nous assumerons pour ce problème que toute tâche commencée par un joueur peut être terminée par l'autre et que toutes les secondes de travail nécessaires peuvent être réparties de n'importe quelle manière entre les deux joueurs, c'est-à-dire que pour les tâches effectuées dans ce problème, 1 joueur

avec 120 secondes est virtuellement indistinguable de 2 joueurs avec 60 secondes. Nous assumerons aussi que, similairement à overcooked, l'ordre dans lequel les tâches sont réalisées n'est pas important.

Il ne reste qu'à calculer le profit généré par recette selon le degré de préparation de chaque ingrédient et le nombre total d'ingrédients à l'aide de la formule suivante:

$$profit = (n_{total\ ingrédients} + n_{non\ préparés} + 2n_{coupés} + 4(n_{cuits} + n_{bouillis})) * \frac{n_{total\ ingrédients}}{3}$$

Spécifications d'entrée:

En entrée vous recevrez:

- **timer:** un *int* correspondant au temps imparti en secondes pour le niveau simulé
- **recipe:** un *array* de *strings* correspondant aux ingrédients avec leurs méthodes de préparation respectives à intégrer dans la recette du niveau simulé

Spécifications de sortie:

En sortie vous devrez fournir:

- **profit:** un entier correspondant au profit total généré par la vente de recettes dans le niveau simulé. **Si vous avez obtenu un float pour le profit total du niveau, il en revient à vous de l'arrondir à l'unité près et de le convertir en entier.**

Exemple d'entrée:

```
240, ["bread", "cut lettuce", "cut tomato", "cooked beef"]
300, ["cut carrot", "cut cabbage"]
265, ["pepper", "cut cheese", "cut tomato", "boiled pasta",
"boiled sauce", "cooked beef"]
150, ["cooked sausage", "cooked bread"]
407, ["boiled rice", "boiled carrot", "cooked beef", "cooked
onion"]
```

Exemple de sortie:

```
624
216
1610
140
1547
```

Explication de la première sortie:

Ceci est la recette d'un burger! Il faut commencer par veiller à la cuisson et par déterminer combien de boulettes de bœuf nous pouvons cuire dans les poêles. Chaque poêle peut cuire $240/7=34,28\dots \Rightarrow 34$ boulettes de bœuf. Avec deux poêles travaillant en même temps, ça nous fait un maximum de 68 boulettes. Pour ce qui est des joueurs, la recette nécessite la seconde dédiée à l'assiette, 1 seconde supplémentaire pour y mettre le pain, 10 secondes pour couper la laitue et la tomate, et 1 dernière seconde pour y ajouter le bœuf cuit pour un total de 13 secondes. En utilisant 240 secondes pour 2 joueurs, les joueurs peuvent assembler un maximum de $480/13=36,92\dots \Rightarrow 36$ recettes dans le temps imparti. Comme nous avons assez d'ingrédients cuits pour toutes ces recettes, c'est notre nombre final! En utilisant la formule pour les profits générés, nous obtenons $(4+1+2*2+4*1)*4/3=17,33\dots$ par burger, pour un total de $36*17,33\dots=624$ profits qu'il est possible de générer dans ce niveau!

VG2: Collisions (35 points)

Dans le monde des jeux vidéos, un élément fondamental calculé par les moteurs de jeux est la collision entre deux objets. Souvent les objets d'un jeu sont composés de cubes, de sphères ou d'autres solides de base.

Dans ce problème, vous devrez déterminer si deux objets sont en collision ou non. Vous recevrez une sphère et un cube. Pour le cube, son orientation sera toujours parallèle à tous les axes. Bref, il n'y a aucune rotation appliquée au cube. S'il était au sol, il serait à plat. Pour la sphère, il n'y a pas de rotation à prendre en compte non plus pour les collisions parce que c'est une sphère...

Pour la sphère et pour le carré, vous recevrez **la position en 3D du centre de l'objet** suivi de la dimension de l'objet. Pour la sphère, la dimension donnée sera le rayon. Pour le cube, la dimension donnée sera la longueur d'un des côtés.

Petit rappel géométrique, pour la sphère, le volume est tout ce qui se trouve à l'intérieur du rayon (inclusivement) à partir du centre de l'objet. Pour le carré, si on part du coin, le volume est tout ce qui se trouve dans une longueur de côté de distance pour chacun des trois axes. Attention, ceci est la logique à suivre à partir d'un coin alors que vous recevrez le centre du cube comme information!

Petit rappel de géométrie de la distance en 3D:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

Spécifications d'entrée:

En entrée vous recevrez:

- **sphere:** un *tuple* de 4 *int*, qui sont les coordonnées en 3D du centre suivi du rayon
- **cube:** un *tuple* de 4 *int* qui sont les coordonnées en 3D du centre suivi de la longueur du côté

Spécifications de sortie:

En sortie vous devrez fournir:

- un *bool* de s'il y a collision ou non

Exemple d'entrée:

sphere = (4, 5, 6, 2)

cube = (6, 6, 6, 2)

```
sphere = (11, 14, 3, 3)
cube = (9, 12, 11, 6)
```

```
sphere = (0, 0, 0, 10)
cube = (0, 9, 14, 10)
```

Exemple de sortie:

True

False

True

Explication de la première sortie:

Dans le premier exemple, nous avons le cube avec son centre à la position $x=6$, $y=6$, $z=6$ et une longueur de côté de 2. Comme c'est un cube de côté 2 et que nous avons le centre, nous allons de chacun des côtés de ± 1 pour trouver nos coins.

Nous obtenons comme cube les 8 coins étant

[5, 5, 5], [5, 5, 7], [5, 7, 5], [7, 5, 5], [5, 7, 7], [7, 5, 7], [7, 7, 5], [7, 7, 7]

Un des coins les plus proches du centre de la sphère est le coin [5, 5, 5]. On peut calculer la distance avec le centre de la sphère (4, 5, 6) et voir si elle est moins grande que le rayon de la sphère pour savoir s'il y a collision.

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

$$d = \sqrt{(4 - 5)^2 + (5 - 5)^2 + (6 - 5)^2}$$

$$d = \sqrt{(-1)^2 + 0^2 + 1^2}$$

$$d = \sqrt{2}$$

Comme la racine carrée de 2 est moins grande que 2 nous avons une collision entre le cube et la sphère. La réponse est donc "True" et nous avons une collision.

VG3: Pac-Man (40 points)

Un des jeux vidéo les plus célèbres de tous les temps est l'iconique jeu de Pac-Man. Le principe du jeu est relativement simple. Pac-Man en jaune doit se déplacer et récolter le plus de points possible sur une carte de jeu tout en évitant des fantômes qui lui courent après.

Vous devrez trouver lequel des fantômes est le plus proche d'attraper Pac-Man sur une grille donnée. Pour calculer la distance, il faut que ce soit la distance la plus courte entre chaque fantôme et Pac-Man. Les fantômes se déplacent de manière orthogonale (haut, bas, gauche, droite) et chaque case à parcourir pour atteindre Pac-Man compte comme une distance de 1.

Dans ce problème, les murs seront des "#", les cases valides pour se déplacer seront des ".", Pac-Man sera représenté par la lettre "P" et, finalement, les fantômes seront représentés par les nombres 1 à 4 pour les 4 fantômes qui poursuivent Pac-Man.

Spécifications d'entrée:

En entrée vous recevrez:

- **grid:** un *array* de *string* qui forme le niveau

Spécifications de sortie:

En sortie vous devrez fournir:

- un *int* représentant le fantôme le plus proche de Pac-Man

Exemple d'entrée:

```
"#####",
"#.....#.....#",
"#.####.####.#.####.####.#",
"#.####.####.#.####.####.#",
"#.....1.....#",
"#.####.#.#####.#.####.#",
"#.....#.....#.....#.....#",
"#####.####.#.####.####.",
"#####.#.P.....#.####.",
"#####.#.####.####.#.####.",
"#.....##.....##.....#",
"#####.#.#####.#.####.",
"#####.#.....4#.####.",
"#####.#.#####.#.####.",
"#..3.....#.....#",
"#.####.#.####.####.#.####.#",
"#....#.....#....#",
"####.#.#.#####.#.#.####",
"#.....#.....#.....#.....#",
"#.#####.#.#####.#",
"#.....2....#",
"#####"
```

```
"#####",
"#..4.....#.....#",
"#.####.####.#.####.####.#",
"#.####.####.#.####.####.#",
"#.....#.....#.....#",
"#.####.#.#####.#.####.#",
"#.....#.....#.....#1.....#",
"#####.####.#.####.####.",
"#####.#....P.....#.####.",
"#####.#.####.####.#.####.",
"#.....##.....##.....#",
"#####.#.#####.#.####.",
"#####.#....2.....#.####.",
"#####.#.#####.#.####.",
"#.....#.....3#",
"#.####.#.####.####.#.####.#",
"#....#.....#....#",
"####.#.#.#####.#.#.####",
"#.....#.....#.....#.....#",
"#.#####.#.#####.#",
"#.....#.....#.....#",
"#####"
```

3

2

On calcule la distance du plus court chemin entre chacun des fantômes et Pac-Man et on choisit celui avec la plus petite distance.

[illegible]

La distance entre le fantôme 2 et Pac-Man est de 30 cases.

La distance entre le fantôme 4 et Pac-Man est de 13 cases.

Donc, comme le fantôme 3 est le plus proche, on retourne 3.

VG4: Tetris (45 points)

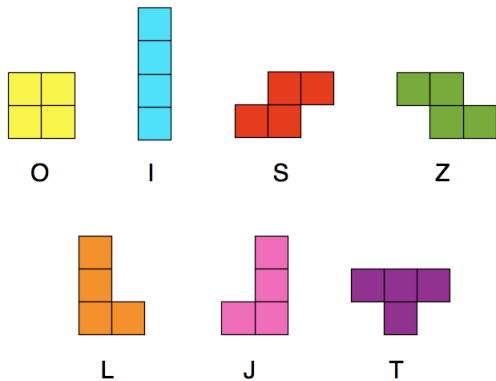
Le but du jeu de Tetris est d'empiler des formes les unes par dessus les autres en essayant de compléter des rangées pour les éliminer et rapetisser la taille de la pile. Il existe plusieurs stratégies pour optimiser son score mais tout part du principe que le joueur sait où la pièce s'arrête. Heureusement pour le joueur, la plupart des jeux de Tetris affichent la position finale de la pièce. Ce sera votre travail de recréer cette fonctionnalité. Je vous rappelle qu'une pièce s'arrête dès que :

- elle touche le bas du plateau
- elle entre en collision avec un bloc existant

Spécifications d'entrée :

En entrée vous recevrez:

- **board**: une *liste* 10x8 remplies de 0 et 1 pour représenter les cases déjà occupées
- **piece**: une lettre majuscule qui représente la pièce à placer dans l'orientation ci-dessous
- **position**: un *int* entre 1 et 10 qui représente la première colonne de gauche à droite dans laquelle la pièce de jeu apparaît



Spécifications de sortie :

En sortie vous devrez fournir:

- **Board**: la même *liste* que donnée en entrée, mais incluant la pièce maintenant placée et dont ses carrés sont désignés par des "x"

Exemple d'entrée :

```
1. Board = [  
  [0,0,0,0,0,0,0,0,0,0],  
  [0,0,0,0,0,0,0,0,0,0],  
  [0,0,0,0,0,0,0,0,0,0],  
  [0,0,0,0,0,0,0,0,0,0],  
  [0,0,0,0,0,0,0,0,0,0],
```

```

[0,0,0,0,0,0,0,0,0,0],
[1,0,0,0,1,0,1,1,1,1],
[1,1,1,0,1,1,1,1,1,1]]
Piece = 'I'
Position = 6
2. Board = [
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,1,1,0,0,0],
[1,1,0,0,0,1,1,1,1,1],
[1,1,1,0,1,1,1,1,1,1]]
Piece = 'T'
Position = 4
3. Board = [
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[1,1,1,0,0,0,0,0,0,0],
[1,1,1,1,0,0,1,0,0,1],
[1,1,1,1,1,1,1,0,0,1]]
Piece = 'O'
Position = 8
4. Board = [
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[0,0,0,0,0,0,0,0,0,0],
[1,0,0,0,0,0,0,0,0,1],
[1,1,0,0,1,1,0,0,1,1],
[1,1,1,0,1,1,1,1,1,1]]
Piece = 'S'
Position = 3

```

Exemple de sortie :

```
1. Board = [
  [0,0,0,0,0, 0 ,0,0,0,0],
  [0,0,0,0,0, 0 ,0,0,0,0],
  [0,0,0,0,0, 0 ,0,0,0,0],
  [0,0,0,0,0,"x",0,0,0,0],
  [0,0,0,0,0,"x",0,0,0,0],
  [0,0,0,0,0,"x",0,0,0,0],
  [1,0,0,0,1,"x",1,1,1,1],
  [1,1,1,0,1, 1 ,1,1,1,1]]

2. Board = [
  [0,0,0, 0 , 0 , 0 ,0,0,0,0],
  [0,0,0, 0 , 0 , 0 ,0,0,0,0],
  [0,0,0, 0 , 0 , 0 ,0,0,0,0],
  [0,0,0, 0 , 0 , 0 ,0,0,0,0],
  [0,0,0,"x","x","x",0,0,0,0],
  [0,0,0, 0 ,"x", 1 ,1,0,0,0],
  [1,1,0, 0 , 0 , 1 ,1,1,1,1],
  [1,1,1, 0 , 1 , 1 ,1,1,1,1]]

3. Board = [
  [0,0,0,0,0,0,0, 0 , 0 ,0],
  [0,0,0,0,0,0,0, 0 , 0 ,0],
  [0,0,0,0,0,0,0, 0 , 0 ,0],
  [0,0,0,0,0,0,0, 0 , 0 ,0],
  [0,0,0,0,0,0,0, 0 , 0 ,0],
  [1,1,1,0,0,0,0, 0 , 0 ,0],
  [1,1,1,1,0,0,1,"x","x",1],
  [1,1,1,1,1,1,1,"x","x",1]]

4. Board = [
  [0,0, 0 , 0 , 0 ,0,0,0,0,0],
  [0,0, 0 , 0 , 0 ,0,0,0,0,0],
  [0,0, 0 , 0 , 0 ,0,0,0,0,0],
  [0,0, 0 , 0 , 0 ,0,0,0,0,0],
  [0,0, 0 , 0 , 0 ,0,0,0,0,0],
  [1,0, 0 ,"x","x",0,0,0,0,1],
  [1,1,"x","x", 1 ,1,0,0,1,1],
  [1,1, 1 , 0 , 1 ,1,1,1,1,1]]
```

Explication de la deuxième sortie :

Premièrement, il faut placer le 'T' correctement horizontalement. La position 4 veut dire que le bloc le plus à gauche du bloc 'T' doit se retrouver dans la colonne 4. Une fois la pièce alignée, on trouve quelle colonne heurte la pièce en premier en la laissant "tomber" et on le place la pièce à cette hauteur.

TRAITEMENT DE TEXTE (TP)

TP1: Rangées et colonnes (40 points)

Une des méthodes d'encryption utilisées durant les guerres mondiales est l'encryption par colonne. Cette méthode consiste à mettre les lettres dans une grille puis à faire la lecture du contenu dans les colonnes plutôt que de gauche à droite comme un texte classique.

Donc, chaque message reçu doit être rangé dans une grille pour être lu par la suite. La portion complexe était donc de trouver combien de colonnes doivent être utilisées pour former la grille qui décode le message secret. Pour ce problème, nous allons choisir le nombre de colonnes pour avoir le nombre le plus égal possible de colonnes au nombre de rangées. Nous allons donc devoir trouver les facteurs du nombre de caractères du message qui sont le plus égaux entre le nombre de lignes et de colonnes. Nous allons aussi considérer qu'il est possible qu'un espace soit laissé vide à la fin de la grille. Ainsi, la grosseur de la grille est soit exactement le nombre de caractères du message (incluant les espaces et signes de ponctuation) ou la longueur du message et une case finale de plus.

Pour certains exemples, il ne sera pas possible d'avoir une grille ayant le même nombre de lignes que de colonnes. Dans ce cas, il faudra avoir plus de colonnes que de rangées.

Pour un message encrypté de 48 caractères, on peut diviser en une grille de 6x8 pour arriver au nombre exact. Par contre, on peut aussi laisser un espace vide à la fin pour obtenir une grille carrée de 7x7 qui va être la solution à privilégier car elle permet d'avoir une grille avec le même nombre de rangées que de colonnes.

Voici quelques exemples de plus sur le choix des dimensions de la grille:

- Pour un message de 30 caractères, on peut l'organiser comme 3x10 ou 6x5, mais on préfère l'organiser dans la grille la plus carré possible pour choisir la grille 6x5.
- Pour un message de 29 caractères, la meilleure option serait d'ajouter un espace vide puis faire une grille de 6x5.
- Pour 24 caractères, nous avons 6x4, mais avec un espace vide supplémentaire, il est possible de faire une grille 5x5 ce qui est à choisir comme option.

Spécifications d'entrée:

En entrée vous recevrez:

- *text*: une *string* du texte encrypté.

Spécifications de sortie:

En sortie vous devrez fournir:

- une *string* qui est le texte décrypté.

Exemple d'entrée:

C wc xeyaalfspryao shessrpie s rnraaitogsl cec

Blssrreatya g pvmedtoe éé sec!

A pgaha eicrny!

Exemple de sortie:

Cyphers are always a classic for text processing

Bravo le message est décrypté!

Again a cypher!

Explication de la première sortie:

Dans le premier exemple, nous avons le message crypté suivant:

C wc xeyaalfspryao shessrpie s rnraaitogsl cec

Ce message contient un total de 48 caractères, ce qui peut être divisé en 8x6, mais si on considère l'option d'avoir un espace vide à la fin du message, on peut se retrouver avec une grille 7x7. Comme les deux côtés ont la même dimension, on préfère la grille 7x7 pour décrypter le message. Nous obtenons donc la grille suivante:

C		w	c		x	e
y	a	a	l	f	t	s
p	r	y	a	o		s
h	e	s	s	r	p	i
e			s		r	n
r	a	a	i	t	o	g
s	l		c	e	c	

De cette grille, nous pouvons extraire le message en lisant les colonnes plutôt que de gauche à droite comme du texte normal. Le message décrypté est donc:

"Cyphers are always a classic for text processing"

TP2: Haïku (20 points)

Les *Haïku* sont des poèmes japonais organisés en 3 lignes séparées par le nombre de syllabes. Dans le cadre de ce problème, les haïkus seront séparés par le nombre de lettres.

Un mot ne peut pas être séparé sauf s'il a un signe de ponctuation comme une apostrophe ou un trait d'union.

Pour faire un haïku dans ce problème, il faut 10 lettres dans la première ligne.

Pour la deuxième ligne, il faut 7 lettres.

Et pour la troisième et dernière ligne, il faut 13 lettres.

Spécifications d'entrée:

En entrée vous recevrez:

- **haïku:** Une *string* du message à essayer de diviser

Spécifications de sortie:

En sortie vous devrez fournir:

- UNE *string* qui est soit "Impossible" si on ne peut pas mettre en forme de haïku ou sinon le haïku bien formaté avec des `\n` pour indiquer les changements de ligne et aucun espace superflu

Exemple d'entrée:

At the CRC we never give up and lose the game!

Un bon robot ne gagne que facilement!

It is hard to write on limited length.

Un arc-en-ciel est magnifique et impressionnant

Exemple de sortie:

Impossible

Un bon robot
ne gagne
que facilement!

It is hard to
write on

limited length.

Impossible

Explication de la première sortie:

Dans la première phrase, les mots “At the CRC we” ont 10 lettres donc la première ligne est bonne! Ensuite “never give” est 9 lettres et non 7, donc c'est impossible de faire un haïku dans le format donné comme on ne peut pas diviser entre deux lettres dans le même mot..

TP3: Palindroming (55 points)

Vous adorez les palindromes, c'est-à-dire les mots, chiffres, phrases ou autres séquences de symboles qui se lisent de la même manière dans les deux sens, comme « madam » ou « racecar ». Vous les trouvez fascinants et vous aimez les créer.

Vous disposez d'un ordinateur qui peut vous aider à traiter les palindromes. Cependant, son fonctionnement nécessite d'énormes ressources, vous souhaitez donc minimiser le nombre d'exécutions.

Votre tâche est simple : on vous donne un string aléatoire s , et vous souhaitez la transformer en palindrome. Vous pouvez soit supprimer des caractères du string, ce qui vous coûtera 1 unité de ressources par caractère supprimé, soit simplement permuter les caractères du string. Si vous souhaitez permuter deux caractères séparés par une distance d , cela vous coûtera d unités de ressources (par exemple, permuter les caractères à l'index 2 et à l'index 5 coûte 3 unités).

Votre objectif est de trouver la quantité minimale de ressources nécessaires pour transformer le string s en un palindrome.

Spécifications d'entrée:

En entrée, vous recevrez:

- un *string*, composée uniquement de caractères ASCII minuscules (a-z), comme seule entrée.

Spécifications de sortie:

En sortie vous devrez fournir:

- un seul *int*, représentant la quantité minimale de ressources nécessaires pour transformer la chaîne en palindrome.

Exemple d'entrée:

crc

mrgoosrem

amanbaplnaacqsanalpwamana

Exemple de sortie:

0

3

7

Explication de la première sortie:

Notez que le string « crc » est déjà un palindrome, donc aucune ressource n'est nécessaire pour la transformer.

Explication de la deuxième sortie:

Dans le string « mrgoosrem », nous pouvons d'abord supprimer le caractère *g* à l'index 2 et le caractère *e* à l'index 7, ce qui nous coûtera 2 unités de ressources et donnera « mroosrm ». Ensuite, nous pouvons permuter les caractères *o* à l'index 3 et *s* à l'index 4, ce qui nous coûtera 1 unité de ressources et donnera « mrosorm », ce qui est un palindrome. Le coût total est de $2 + 1 = 3$ unités de ressources.

TP4: Pangramme (35 points)

Un pangramme est une phrase qui contient toutes les lettres de l'alphabet. Les plus intéressants sont ceux avec le moins de répétitions possibles. Vous devrez faire un programme qui détecte si une phrase est un pangramme et si c'est le cas, vous devrez fournir toutes les lettres répétées dans l'ordre dans lequel elles ont été répétées.

Spécifications d'entrée:

En entrée, vous recevrez une *string* qui est la phrase qu'il faut étudier pour voir si elle est un pangramme.

Spécifications de sortie:

En sortie vous devrez fournir:

- une string qui est vide si la phrase n'est pas en pangramme et contient sinon dans l'ordre toutes les lettres répétées.

Exemple d'entrée:

The quick brown fox jumps over the lazy dog

Sphinx of black quartz, judge my vow

aaaaaabcdefghijklmnopqrstuvwxyz

Portez ce vieux whisky au juge blond qui fume

Exemple de sortie:

"ouoerteo"

"auo"

""

"eeiueouiue"

Explication de la première sortie:

Dans la phrase "The quick brown fox jumps over the lazy dog", qui est le pangramme le plus connu, nous avons toutes les lettres de l'alphabet présentes. Ainsi, nous devons trouver les doublons. Dans ce cas, les doublons identifiés sont principalement des voyelles ainsi que r et t. Remarquez que les lettres ne sont indiquées que pour leur répétition et dans l'ordre ou la répétition est rencontrée.